

Hi-Tec Shopify Automation

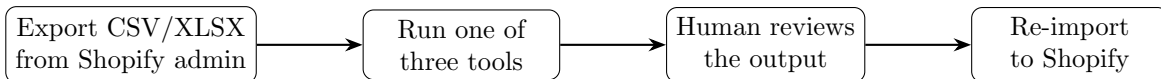
Brief

Explainer generated for portfolio review

Covers the code as it exists in the repository, 2023 client work by Salman Adnan

1 What it is

Hi-Tec Bearings runs a Shopify store but kept warehouse stock records (quantities, costs) in a separate spreadsheet from the Shopify product catalogue, so keeping the two in sync by hand was slow. This is three small Python tools, by one author, that automate that reconciliation by working on the CSV/XLSX files exported from and re-imported into the Shopify admin panel — deliberately never touching Shopify’s API. That keeps a human reviewing every catalogue change before it goes live, at the cost of having to match records on text fields (titles, variant values) instead of a stable product ID.



2 The three tools

`quantity_pricing_updater/`

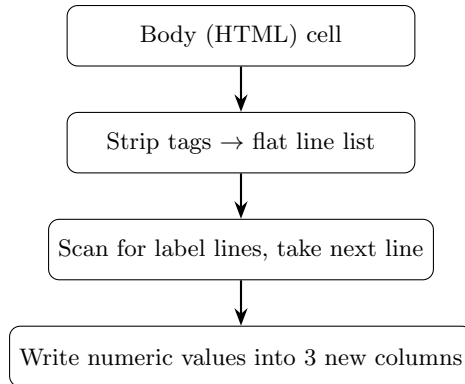
Merges the client’s warehouse stock list into a Shopify export. Matches single-variant products by **Title** and variants by **Option1 Value**, against stock-list rows filtered to **Brand == 'HI-TEC'** first (the same item code can appear under other brands too, and only the HI-TEC row is trustworthy). Adds **Updated Quantity** and **Updated Cost** columns and writes `updated_<name>.csv`.

`dimensions_extractor/`

Shopify product descriptions embed a bearing-dimensions table as raw HTML. This tool strips the HTML tags, splits the remaining text into lines, and scans for the dimension labels (bore diameter, outside diameter, width), taking the value on the following line. Writes the three numbers into new columns so they can become Shopify metafields (filterable product attributes).

`stock_updater_gui/`

The version handed to the client: a tkinter desktop app with three buttons wrapping both tools above plus a third feature, variant detection, which parses bearing suffixes (K, E, 2RS, Z, ...) out of variant names into **Bore**, **Case**, and **Seal Type** columns. Accepts CSV or XLSX input, always writes XLSX, and never overwrites an existing output file (it appends an incrementing counter to the filename instead).



3 Notable design decisions

- **Brand filter before matching.** Filtering the stock list to HI-TEC before joining on item code removes an entire class of wrong matches that arise because the same item code can appear under a competing brand's row too.
- **Human-in-the-loop by design.** No Shopify API access; every tool round-trips through a spreadsheet a person reviews, trading automation depth for a manual safety check.
- **Packaging for a non-technical client.** The CLI tools ship with PyInstaller .spec files to build standalone Windows executables; the GUI wraps all three operations behind buttons with file pickers and a non-destructive save.

4 Known limitations (from the author's own retrospective)

- The matching logic is duplicated almost line-for-line between the two CLI scripts and the GUI script, rather than living in one shared module.
- Matching is done with per-row Python loops ($O(n \times m)$); a single pandas `merge` on `Item/Brand` would be both correct and much faster at the client's real scale (roughly 1,600 stock rows against an 11,000-line Shopify export).
- The code writes into filtered DataFrame slices in a way that triggers a pandas ambiguous-copy warning (`SettingWithCopyWarning`); this is why `requirements.txt` pins pandas below version 3 — pandas 3.0's copy-on-write default would turn that warning into silently lost writes.
- The dimensions extractor assumes a strict label-line-then-value-line ordering in the HTML, which matches this store's template but is not a general HTML table parser and would silently miss dimensions elsewhere.
- Debug `print` statements remain from development, and there is no automated test suite.

5 Sample data note

The original repository contained the client's real data (about 1,600 stock rows and an 11,000-line Shopify export); that was client-confidential and has been replaced in this portfolio copy with a small fabricated sample of the same shape. Both CLI tools were re-verified against that sample;

the GUI was only smoke-tested to confirm its window opens. A full live-store run (export, process, re-import) was done during the original 2023 engagement and was not repeated for this write-up.

6 Glossary

API A defined way for one program to talk to another automatically.

CSV / XLSX

Plain-text and Excel spreadsheet file formats.

DataFrame

pandas' in-memory table object (rows and named columns).

Metafield Shopify's mechanism for attaching extra, filterable structured data to a product.

PyInstaller

Bundles a Python script and its dependencies into one standalone executable.